

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ARQUITECTURA: Single-File Hybrid (2 archivos máximo)

Patrón arquitectónico seleccionado: Micro Frontend + Micro Backend desacoplados, comunicados via HTTP REST local.

FRONTEND (index.html):

- Arquitectura monolítica de archivo único con separación lógica interna por secciones: HEAD (meta + CDN Tailwind), BODY (estructura HTML semántica), STYLE (overrides CSS mínimos), SCRIPT (lógica JS vanilla).
- Estado de UI manejado de forma imperativa directa sobre el DOM sin frameworks reactivos, dado el alcance mínimo de la herramienta.
- Flujo de interacción: INPUT !' VALIDACIÓN LOCAL !' FETCH POST !' LOADING STATE !' RENDER RESULTS.
- Validación client-side: detección de textarea vacío con feedback visual (border-red-500 de Tailwind), bloqueo de petición de red.
- Manejo de estados de UI: idle, loading (botón deshabilitado + texto 'Analizando...'), success (panel de resultados visible), error.
- Comunicación con backend via fetch() nativa con async/await, Content-Type application/json.
- Lógica de renderizado condicional del panel de resultados y cálculo de estado 'Óptimo' vs 'Alerta' basado en umbral de 1000 tokens.

BACKEND (main.py):

- Framework: FastAPI (Python) por su soporte nativo async/await, tipado con Pydantic y rendimiento ASGI.
- Middleware CORS configurado para orígenes: http://localhost, http://127.0.0.1, null (file://) para permitir apertura directa del HTML desde sistema de archivos.
- Endpoint único: POST /api/analyze con modelo Pydantic de request (prompt: str) y response (tokens_estimados: int, latencia_ms: int, estado: str).
- Lógica de negocio: cálculo de tokens = round(len(prompt.split()) * 1.3), latencia simulada = random.randint(500, 2000), asyncio.sleep(latencia/1000) no bloqueante.
- Determinación de estado: 'Óptimo' si tokens <= 1000, 'Alerta' si tokens > 1000.
- Servidor ASGI: Uvicorn como runner recomendado.

SEGURIDAD:

- CORS restringido a orígenes locales únicamente, no wildcard en producción.
- Validación de payload con Pydantic evita inyección de tipos inesperados.
- Sin persistencia de datos, sin autenticación requerida (herramienta interna local).
- Sanitización implícita al operar solo con conteo de palabras, sin ejecución de contenido del prompt.

DEPENDENCIAS BACKEND: fastapi, uvicorn, pydantic (incluido con fastapi).

DEPENDENCIAS FRONTEND: Tailwind CSS via CDN (sin instalación local).

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 14,542

Tokens de salida (output): 6,957

Total tokens: 21,499

Horas-hombre estimadas: ~2.4 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

Arquitecto (default): !"1,080 input | !'894 output | 1 llamadas

Tech Lead (default): !"1,668 input | !'2,999 output | 2 llamadas

Desarrollador (default): !"4,619 input | !'2,438 output | 2 llamadas

QA (default): !"7,175 input | !'626 output | 2 llamadas